

AI and Adaptive Systems

Harry Morrison

May 2025

1 Deterministic

1.1 Direct Methods

Seek optimum by sampling, you learn a bit about the space each time

- Cyclic
 - Cyclic Coordinate Search: Series of line optimisations, line search can be any way (golden section/fibonacci)
 - Cyclic Coordinate Search with Acceleration: Assume net progress, direction after the first n step cycle is given by the vector difference
 - Powells Method: Uses basis vectors which are stored in a queue, search along those basis vectors
- Pattern
 - Hooke-Jeeves: Searches in grid, $(2n)$ searches per point
 - Generalised Pattern Search: $(n+1)$ searches per point, can generalise using a set of directions
 - Nelder-Mead Simplex: Works in multiple dimensions, has highest, second highest, and lowest vertices of all vertices in simplex, and mean of points centroid, α (reflection) and β (expansion) and γ (contraction) and σ (shrinkage), Use variance of vertex values as a proxy for curvature, collective intelligence, $(n+1)$ vertices and centroid (x_h)
- Simplex

2 Stochastic

- Hill Climbing
- Hill Climbing Steepest Ascent
- Hill Climbing Steepest Ascent with Replacement

2.1 Single State

- Hill Climbing with Restarts: Short time intervals weighted toward exploration, long time intervals weighted toward exploitation
- Simulated Annealing
- Tabu Search: Discrete Spaces, tabu list of where its been, really a queue
- Iterative Local Search: Using heuristics about where to restart, assumes better local optima can be found near the one your in

Tweaks: - Gaussian: many samples and low sigma means exploitative, few samples and high sigma means exploration - Uniform - Custom

3 Population Based

3.1 Background

Traditional search spaces fail dismally:

- Hill Climb fails in "High modal" spaces
- Brute force (grid search), breadth first, and variants are infeasible in real-valued, high dimensional spaces
- Evaluations are expensive even when a single one is cheap, we need a lot of them

Key ideas:

- We want to take into account every evaluation $\rightarrow$ gives information about search space
- So can a collection of points tell us more together than each individual on its own?

Inspiration - Natural biological systems:

- Adapting to perform better within an environment
- Darwin's theory of evolution

Typical Steps:

1. Create (random) initial population
 - BuildInitialPopulation()
 - Uniform distribution $\rightarrow$ does that require the space to be bounded?
 - Infinite distribution (Gaussian) $\rightarrow$ where do you centre it? How does that bias result?
 - Can be intentionally biased if you know something about the space.
2. Assess/evaluate fitness (quality)
 - AssessFitness()
3. "Breed" new population of offspring
 - Breed() $\rightarrow$ Involves selection and "tweaking" (mutation and/or recombination)
4. "Join" from parents and children to form next generation
 - Replace parents with offspring, or keep some parents (selection)

3.2 Evolutionary Strategies

- μ is the number of surviving parents
- λ is the number of offspring
- Randomly initialise with λ individuals

AssessFitness() evaluate and keep the μ fittest (truncation selection)

Breed() mutate each of these μ parents $\frac{\lambda}{\mu}$ times to get λ children

Join() replace the parents with the children, repeat.

3.2.1 (μ, λ)

λ :

- Similar to n in steepest ascent
- Bigger would tend to be better, if not for cost!
- Infinite is just random search

μ :

- How selective the algorithm is
- Low μ , more exploitation

Mutate() $\rightarrow$ probability and degree of mutation also affect exploration/exploitation (not independent, highly random mutation with small selection still effectively a random walk).

3.2.2 $(\mu + \lambda)$

Join() operation is the only difference with (μ, λ) . Keeps μ fittest parents

3.2.3 Comparisons

(μ, λ) VS $(\mu + \lambda)$

- In $(\mu + \lambda)$ offspring compete with parents for place in next generation
 - $(\mu + \lambda)$ tends to be more exploitative than (μ, λ)
 - * Less chance of losing a good solution found so far
 - * More chance of whole population getting stuck in local optima
- In general, population-based approaches
 - Want to maintain diversity in a population
 - Accept reduction in diversity over time to encourage convergence on a solution (exploitation)
 - Premature convergence
 - * Too much loss of diversity too soon
 - Not enough exploration

ES VS Single-State Methods

- many single-state methods are specialisations (or degenerate cases) of population-based methods
- Or to put it another way, population-based methods are generalisations of single-state methods
 - Hill Climb $(1 + 1)$
 - Steepest ascent hill climb $(1 + n)$
 - Steepest ascent hill climb with replacement $(1, n)$

3.3 Adaptive Mutation

- **Typical use:**
 - fixed-length vector of real-value numbers ("Chromosome")
 - Mutation performed using "Gaussian Convolution" (Alg.11)

- Recall Gaussian mutation controlled by σ
- called the **mutation rate** of ES
- How do you choose mutation rate? You can guess, run experiments to find a good value, run a meta-optimisation, decrease value over time (simulated annealing), adaptively change based on some statistic(s) of the system...
- **Example (Rechenberg)**
 - “One-Fifth Rule”
 - * Derived from experiments on simple test problems (your problem may differ!)
 - * Mutation rate adjusts over time...
 - If $> \frac{1}{5}$ of children are fitter than their parents, then we’re exploiting local optima too much
→ increase σ^2
 - If $< \frac{1}{5}$ of children are fitter than their parents, then we’re exploring too much
→ decrease σ^2
 - If exactly $\frac{1}{5}$ of children are fitter than their parents, don’t change anything.
- **Self-Adaptive Mutation**
 - Each individual might have its own mutation operator
 - Mutation operators might mutate themselves
 - Occam’s Razor
 - * Complexity for its own sake is poor practice
 - * Must empirically demonstrate the benefits

3.4 Evolutionary Programming

- Extra constraints are often required to ensure individuals are viable solutions
- Depends on representation: Finite-state automata, trees, graphs, programs, neural network structures

3.5 Genetic Algorithms

- Inspired by Darwinian model of evolution
- Holland 1970s
- Selection, Crossover, Mutation
- Similar to (μ, λ) ES but focus on individual vector as chromosome/genome
- crossover is fundamental with many approaches

3.5.1 Mutation

One-Point Crossover Disadvantages

- Highly likely that two units far from each other will be separated
- Unlikely that two units close to each other will be separated

- Linkage of "genes" could be great between some genes

Two-Point Crossover Disadvantages

- Long distances still more likely to be broken than short ones
- Chunks that are being swapped and broken up may have some sort of meaning in the hypothesis space, which could make fitness of child worse

Disadvantages Uniform Crossover Disadvantages

- If two parents have the same value in a spot, it can never be restored
- In a hypercube, crossover alone can only produce children on corners of the same hypercube
- Any time all parents have one gene the same, the hypercube collapses in that dimension, thus can never be restored
- Eventually the population will converge to a single point (individual chromosome)
- Usually converge is *premature* (unless it happens to coincide with an optimal solution)

So Why Crossover?: Just because its in biology doesn't make it the best for computer algorithms, so more justification in needed.

Schema/Building Blocks Theory: Highly fit individuals share short, low-order schemata (patterns) (have same traits). Crossover promotes and recombines these "good chunks" through the population.

Assumption of epistasis and Linkage: Relies on the assumption of epistasis (interaction strength among genes) and linkage (probability dependent set survives a crossover)

- One point $\rightarrow$ Survival $\propto$ distance & position
- Two point $\rightarrow$ Survival $\propto$ distance
- Uniform $\rightarrow$ Survival $\propto$ mixing probability p

3.5.2 Selection

Truncation ($\mu + \lambda$)

- keep top candidates (μ), discard rest
- **Complexity:** $O(n \log n)$ sort

Fitness Proportionate Selection

- Select individuals in proportion to their fitness
- Achieved by "sizing" and "stacking" according to fitness
- **Complexity:** $O(n \log n)$
- Let sum of fitnesses $s = \sum_i f_i$
- Simple but sensitive to absolute fitness scaling and close fitness values; high variance

Stochastic Universal Sampling

- Baker 1987
- stack as in fitness proportionate, and select at regular intervals
- Low variance resampling
- Ensures any individual with $f_i \geq \frac{s}{n}$ gets chosen at least once

- Preserves fitter individuals, while retaining some probability of choosing less fit (leans toward exploitation)
- **Complexity:** $O(n)$

Tournament Selection

- A non-parametric selection algorithm only considers the rank order of the individuals
- Not sensitive to size of fitness values
- Simple and easily parallelised
- Tuneable by setting tournament size t
- $t=1$, is random selection
- t large ($t \propto$ pop. size), tournament selection $\rightarrow$ truncation selection with $\mu = 1$

Elitism

- Fittest individual(s) automatically preserved for the next generation
- Favours Exploitation
- Caution: may cause lack of diversity and premature convergence!
 - Must be kept in check
 - increase mutation
 - Breed in diversity from outside elite group
 - Reduce how many elites are stored ($\mu + \lambda$)

4 Particle Swarm Optimisation (PSO)

4.1 Big idea

- Each particle imitates a *bird* in a flock:
 - momentum** keeps the bird's current heading $\Rightarrow$ **exploration**;
 - personal best** pull toward its own best catch $p_{\text{best}} \Rightarrow$ **individual exploitation**;
 - global (or neighbourhood) best** pull toward the flock's best catch $g_{\text{best}} \Rightarrow$ **social exploitation**.
- These simple rules yield an *emergent* drift toward richer fishing grounds (lower-cost regions). **emergent behaviour** of a group over time
- Combine a balance of:
 - Collective behaviour - can promote exploitation?
 - Individual behaviour - can continue exploration?

4.2 Mechanics in symbols

$$\mathbf{v} \leftarrow \alpha \mathbf{v} + \beta r_1 (\mathbf{p} - \mathbf{x}) + \gamma r_2 (\mathbf{g} - \mathbf{x}), \quad \mathbf{x} \leftarrow \mathbf{x} + \epsilon \mathbf{v}, \quad (1)$$

where $r_1, r_2 \sim \mathcal{U}(0, 1)$.

- High $\alpha \Rightarrow$ more exploration (Own particles inertia).
- High γ or a fully connected topology $\Rightarrow$ faster exploitation.
- Friction (velocity damping) helps fine search near optima.

Symbol	Meaning
$\mathbf{x}$	particle position (candidate solution)
$\mathbf{v}$	velocity (momentum)
p	particle's personal best position so far
g	globally / neighbourhood best position
α, β, γ	weights on momentum, pull to p , pull to g
ϵ	step-size (may decay over time)

4.3 Alternative viewpoint

- **Momentum:** Newton's first law (object in motion stays in motion).
- **Attractions** to p and g act like "gravitational" pulls.
- Optional friction damps oscillations, preventing overshoot.

5 Differential Evolution (DE)

- Population of real-valued vectors; no explicit velocity.
- For each target vector choose three distinct others and propose a *mutant*

$$\mathbf{m} = \mathbf{a} + F(\mathbf{b} - \mathbf{c}),$$

where $F \in (0, 2)$ scales the step (larger F = larger exploratory jump).

- **Crossover:** blend $\mathbf{m}$ with the target; child competes with its parent.
- Intuition: the triangle ($\mathbf{a}, \mathbf{b}, \mathbf{c}$) points the fourth corner toward unexplored space; triangles shrink as population converges, giving adaptive step sizes.

6 Hybrid / Memetic Algorithms

- Combine a *global* population-based method (PSO, DE, GA...) with a fast *local* optimiser (hill-climb, gradient step).
- Population finds promising *regions*; local search refines each individual.
- **Memetic** view: individuals "learn" within their lifetime and still pass their genes/memes to offspring.
- Hyper-parameters:
 - frequency of local search;
 - intensity / number of local steps;
 - which subset of individuals to refine (all, top $k\%$, random sample).

7 Exploration vs. Exploitation Summary

Algorithm	More exploration	More exploitation
PSO	high α ; sparse topology; large random factors	high γ ; friction; decaying ϵ
DE	larger F ; high crossover; big population	shrinking F ; elitist selection; local search
Hybrid	postpone / rare local search	frequent, intensive local search on elites

8 Self-check Questions

1. Why does increasing α in PSO slow convergence on unimodal but *help* on multimodal landscapes?
2. In DE, what happens if (a) $F = 0$ or (b) $F > 2$?
3. Sketch a hybrid PSO+hill-climb scheme and name two new hyper-parameters you must tune.
4. Give one pro and one con of “fully connected” vs. “ring” topology in PSO.

8.0.1 Differential Evolution (DE)

8.1 Hybrid Optimisation Algorithms

9 Perceptron to Neuron

9.1 1-d Classifiers

- Inspired by Frank Rosenblatt’s Mk 1 Perceptron
- **Goal:** Learn decision functions (classifiers) from data
- Example used: escaping velocity from moon

9.1.1 Perceptron Model

- A simple linear classifier with threshold:

$$f(x) = \begin{cases} 1, & x > T \\ 0, & x \leq T \end{cases}$$

- Reformulated with bias, where $b = -T$:

$$f(b, x) = \begin{cases} 1, & x + b > 0 \\ 0, & x + b \leq 0 \end{cases}$$

- **Hypothesis Space:**

- Language: Activation function
- Hypothesis/Candidate: value for b
- Hypothesis Space: 1-dimensional space (number line)

- **Cost Function:**

$$C(b) = \frac{1}{2n} \sum_x (y(x) - a(b, x))^2$$

- **Problem with step activation:**

- Heaviside step function
- Not differentiable, so you can’t use gradient descent
- Solution: approximate the threshold function with a continuously differentiable function

9.1.2 Sigmoid Activation Function

- **Properties:**

- Monotonically increasing
- Constrained by horizontal asymptotes
- First derivative is a bell-shaped curve (e.g. normal distribution)
- Removes conditional part of a function
- Is asymptotic so can never output 1, so MSE will never be 0

- **Logistic Function**

- Commonly used sigmoid function

$$\sigma(z) = \frac{1}{1 + e^z}$$

9.1.3 Our Example:

- **Logistic Neuron Activation Function:**

$$a(b, x) = \frac{1}{1 + e^{-(x+b)}}$$

- **Cost Function:**

$$\frac{dC}{db} = -\frac{1}{n} \sum_x \left(y(x) - \frac{1}{1 + e^{-(x+b)}} \right) \cdot \frac{e^{-(x+b)}}{(1 + e^{-(x+b)})^2} \quad (2)$$

$$= -\frac{1}{n} \sum_x (y(x) - a(b, x)) \cdot a(b, x) \cdot (1 - a(b, x)) \quad (3)$$

- Gradient Descent: $\rightarrow x = x - a f'(x)$

- Improvements:

- Add more features (quadratic terms)
- Trade-off: More complexity, less interpret ability or changing to step function can loose gradient method
- Change sigmoid activation function
- Change error metric

9.2 2-d Classifiers

9.2.1 Changing the 1-d Classifier

- Our language was not expressive enough for our hypothesis space and classify our sample data
- Error metric (MSE) - We want a generic approach so we don't add another degree of freedom
- Algorithm (Gradient Descent) - The idea is good
- Sigmoid (Logistic function) - We can change this to make it steeper. Our previous approach made it balance out the errors on both sides. Making it steeper would mean the errors would have great penalty instead of balancing out.
 - Add a weight to the denominator:

$$a(w, b, x) = \frac{1}{1 + e^{-(wx+b)}}$$

- Going to make it more discriminative
- Scaling factor
- Negative: Going to lose some gradient of the slope
- w and b could continuously diverge but it's not a problem if it gives the same answer ($\frac{-b}{w}$)

10 Neurons to Logic - n-D Classifiers

10.1 n-D Hyperplanes

A logistic neuron with weight vector $\mathbf{w}$ and bias b computes:

$$a(w, b, x) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}}$$

- **1 input** → threshold point splits the real line into accept vs reject
- **2 input** → same formula; algebra shows the 0.5-threshold is a line in (x_1, x_2) space.
- **3 input** → threshold is a plane; n inputs → a $(n - 1)$ -dimensional *hyperplane*

The key mental shift: a neuron is always carving the input space in two, no matter how many dimensions.

10.2 Computing Different numbers

Training Neuron on '1'

- Cost dropped smoothly; activations for digit 1 climbed above 0.5 while all others fell below → perfect separation.
- This succeeds from most random starts and in few iterations. Why? Digit 1 lights only two segments, so “penalise lit segments” plus a bias already knocks every competing digit well below the threshold.

Training Neuron on '7': A harder digit

- Digit 7 has three lit segments—very similar to '1'.
- Repeated random inits often fail: gradient descent stalls in local minima; 7's activation can't clear 0.5.
- Lowering the stopping or running 25 k+ iterations still stuck.
- But if you hand-craft the initial weights (reward the three correct segments with +1, punish the four off-segments with -1, bias = -2.5) it learns instantly → proves a valid solution exists.

Key Takeaway:

- the model class cannot represent the function (expressiveness problem), or
- the optimisation algorithm can't find the right weights (search problem). For '7' the obstacle was (2). Recognising this distinction is vital in research and model debugging.

10.3 Generalising - Adequacy of Hypothesis Space

Is a single neuron expressive enough for every digit? The hypothesis space (set of all weight/bias choices) already contains perfect decoders for 0-9. The optimisation challenge remains.

Hyper-cube Slicing :

- Each possible combination of segments (there are 27 of them) is represented by a point in 7-dimensional space

- For any vertex you can draw a hyperplane that "cuts off" just that corner → neuron can accept exactly that pattern.

Hit Counting :

- Score a pattern: +1 per correctly lit segment, -1 per incorrect.
- The target digit will always get the highest score.
- Set bias midway between its score and the runner-up

Neurons as Logic Gates :

- By suitable (integer) weights and bias, a two-input neuron realises AND, OR, NOR, NAND... therefore any Boolean formula over 7 literals.
- Compose the literal tests for "segment i on/off" into a Boolean expression for your digit, collapse it into a single NAND tree, and note that one neuron can implement a NAND of 7 inputs (weights +1, bias -6.5).
- Thus any digit's truth table is realisable.

10.4 Pairs of Digits & the XOR Barrier

Recap:

- Any single digit on a 7-segment display can be isolated by one neuron.
- Geometric lens: choose a hyper-plane that slices off one corner of the 7-D hyper-cube.
- Logic lens: craft an AND of seven literal tests (segment i on/off).
- "Hit-count" lens: reward correct segments, penalise errors, bias midway between best and second-best.

Problem:

- Trying to distinguish between a "4 or 7" from "1 or 9" is impossible with our current logic
- After visualising, "4 or 7" is not representable by a single neuron - the limitation is in the model class, not in gradient descent.
- The **Exclusive OR** is not supported and is a problem.

The XOR problem & historical fallout

- Minsky & Papert (1969) proved that perceptrons cannot realise XOR.
- Crucial nuance: XOR can be expressed by a network of neurons — two hidden units form two half-planes; a third neuron "ANDs" their outputs, giving the XOR region. (See slide figure.)
- Moral for modern ML: when a task isn't linearly separable, widen the hypothesis space (more layers, non-linear features) before you blame the optimiser.

11 Neural Networks

Why Leave Single Neuron Structures

- It can implement any linearly separable pattern (single hyperplane) but cannot solve XOR or anything that reduces to XOR
- It can isolate one digit (or any two adjacent digits) on a 7-segment display but cannot carve out "4 v 7", "even digits" because those need multiple regions

- **Fix:** chain neurons into a network so their half-spaces compose richer regions. With enough hidden units a feed-forward net can approximate any continuous function (universal approximation theorem – proved next lecture).

Biological Inspiration

- Visual cortex is layered ($V_1 \rightarrow V_2 \rightarrow \dots$); early layers fire on simple edges, deeper layers on faces, wheels, etc.
- Principle of increasing abstraction: deeper layers recombine earlier detections into ever more abstract concepts — not just for vision but any modality.

11.1 Architectural Building Block - the Feed-Forward Neural Network

- Has input layer, hidden layers(encoding), and output layer.
- Biases typically not shown
- Neuron outputs all the same (only one per node)
- Sub-symbolic approach: layers and nodes can look at certain nuanced aspects. sub-symbolic representation - "degree of 4-ness", curviness, corners.
- more expressive but harder to find solution
- may not be intuitively meaningful
- less brittle - graceful degradation
- Design dark-art: You pick depth & width; there are guidelines but no universal recipe. Researchers increasingly let evolutionary / swarm optimisers mutate architecture as well as weights ("neuro-evolution").
- Rule based methods failed: Humans vary their strokes in numbers and appearance and exhaustive "if...then" rules become brittle and break. Small noise breaks them

12 Back Propagation & Universal Approximation

12.1 Supervised-learning recap

- **Training set:** input-output pairs $\{(\mathbf{x}^{(s)}, \mathbf{y}^{(s)})\}_{s=1}^S$.
- Task: learn a parametrised function $f(\mathbf{x}; \theta)$ (weights w and biases b) such that $f(\mathbf{x}) \approx \mathbf{y}(\mathbf{x})$.
- *MNIST example:*
 - Input $\mathbf{x} \in \mathbb{R}^{784}$ (a 28×28 greyscale digit, rescaled to $[0, 1]$).
 - Target $\mathbf{y} \in \{0, 1\}^{10}$ (one-hot for the true digit).

12.2 Cost (loss) functions

Per-sample norms

$$L_2(\mathbf{x}) = \sum_{i=1}^N (f_i(\mathbf{x}) - y_i(\mathbf{x}))^2, \quad L_1(\mathbf{x}) = \sum_{i=1}^N |f_i(\mathbf{x}) - y_i(\mathbf{x})|.$$

Mean-squared error over the whole set

$$C(w, b) = \frac{1}{2S} \sum_{s=1}^S \|f(\mathbf{x}^{(s)}; w, b) - \mathbf{y}^{(s)}\|_2^2. \quad (4)$$

$C \geq 0$, differentiable, and $C \rightarrow 0$ iff the network reproduces all targets.

12.3 Training as optimisation

- Collect all parameters into a single vector $\theta = (w, b) \in \mathbb{R}^P$.
- Minimise the cost with (stochastic) gradient descent:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} C(\theta), \quad \eta > 0 \text{ learning-rate.}$$

12.4 Back-propagation algorithm

Forward pass

compute for each layer L

$$z_j^L = W_{j\cdot}^L \mathbf{a}^{L-1} + b_j^L, \quad a_j^L = \sigma(z_j^L).$$

Backward pass

initialise output errors

$$\delta_j^L = \frac{\partial C}{\partial z_j^L} = (a_j^L - y_j^{(s)}) \sigma'(z_j^L)$$

then propagate

$$\boldsymbol{\delta}^{L-1} = (W^L)^\top \boldsymbol{\delta}^L \odot \sigma'(\mathbf{z}^{L-1}).$$

Gradients

$$\frac{\partial C}{\partial b_j^L} = \delta_j^L, \quad \frac{\partial C}{\partial W_{jk}^L} = \delta_j^L a_k^{L-1}.$$

12.5 Smooth vs non-smooth activations

- Back-prop assumes σ is differentiable (sigmoid, tanh, logistic).
- Using a hard step (Heaviside) breaks the gradient; one must then resort to derivative-free optimisers (genetic algorithms, PSO, etc.)—typically slower and less precise.

12.6 Universal approximation theorem

A feed-forward network with a *single* hidden layer containing a finite number of sigmoid (or tanh) units can approximate *any* bounded continuous function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ to arbitrary accuracy $\varepsilon > 0$.

- Guarantees *existence* of suitable weights, not that gradient descent will find them.
- Required width may be huge; deeper networks can reach the same error with fewer neurons.